SESIÓN 11 · REACT GUIDE VI

Una selección.
Un formulario compartido.

Formularios, Zod y modal global

Sesión 11 · MediaLog

De elegir Dune a preparar sus datos.



Zustand

Mantine Form

Zod

MAPA DE LA SESIÓN · ELEGIR → ABRIR → COMPLETAR → VALIDAR → PREPARAR

Tres herramientas, un mismo recorrido



Resolver = puente entre Form y Zod. Si hay errores, corregimos los campos. El objeto final todavía no se envía.

MEDIALOG · ELEGIR

Ya consultamos. Ahora queremos completar.

Al terminar la 10

Búsqueda y detalle consultan el catálogo.

```
movie · 123
```

La URL indica qué medio mostrar.

El nuevo gesto

«Quiero agregar este medio a mi biblioteca».

Agregar a mi biblioteca

Necesitamos estado, prioridad y notas.

Hoy prepararemos el body: el objeto que después enviaremos al servidor.

MEDIALOG · ABRIR

Dos entradas, un mismo formulario

Búsqueda
Elijo una tarjeta.

abre →

Modal compartido
Completo los datos del medio elegido.

abre ←

Detalle
Elijo el medio de esta página.

Modal: una ventana sobre la página actual.

MEDIALOG · ABRIR

Una ficha sobre el mostrador

La ficha

```
movie · 123
```

Indica qué medio vamos a completar.

El formulario

Recoge lo que la persona escribe o elige.

La revisión

Comprueba que esos datos cumplan las reglas.

Zustand comparte la selección · Mantine Form gestiona los campos · Zod valida.

MEDIALOG · ABRIR

Cada dato conserva su responsable

Dato	Responsable	Ejemplo
Dirección de la página	React Router	`/media?mediaType=movie…`
Respuesta del catálogo	TanStack Query	Datos recibidos del servidor
Medio elegido para el modal	Zustand	`selection`
Campos y errores	Mantine Form	`status, priority, notes`
Reglas de esos campos	Zod	Notas: máximo 256 caracteres

MEDIALOG · ABRIR

El store guarda una decisión pequeña

src/stores/create-media-log-modal-store.ts · contrato

```ts
type CreateMediaLogModalStore = {
  selection: GetMediaDetailInput | null

  openCreate: (
    selection: GetMediaDetailInput
  ) => void

  closeCreate: () => void
}
```

Store = estado compartido

Selección más acciones para cambiarla.

`null → cerrado`

`medio elegido → abierto`

En memoria: recargar no conserva esta selección.

No guarda notas, respuestas del servidor ni datos de sesión.

MEDIALOG · ABRIR

Si hay selección, el modal está abierto

Evitar dos decisiones independientes

Estado que puede contradecirse

```
opened: true
selection: null
```

Abierto… ¿para qué medio?

Calcular desde la selección

Dentro de CreateMediaLogModal

```
const opened = selection ≠= null
```

Un solo dato decide ambas cosas.

MEDIALOG · ABRIR

create crea el hook; set cambia el estado

src/stores/create-media-log-modal-store.ts · implementación

```ts
import { create } from 'zustand'

export const useCreateMediaLogModalStore =
  create<CreateMediaLogModalStore>()((set) => ({
    selection: null,

    openCreate: (selection) => set({ selection }),

    closeCreate: () => set({ selection: null }),
  }))
```

Dos operaciones

`openCreate` recibe los datos del medio elegido.

`closeCreate` retira la selección.

`set` actualiza esos campos y conserva las acciones.

No se hace una petición HTTP al abrir el modal.

MEDIALOG · ABRIR

Un selector elige qué leer del store

src/components/create-media-log-modal.tsx · dentro del componente

```
const selection = useCreateMediaLogModalStore(
  (state) ⇒ state.selection
)

const closeCreate = useCreateMediaLogModalStore(
  (state) ⇒ state.closeCreate
)
```

Leer una parte

`state` es el objeto del store.

La función devuelve la parte que este componente necesita.

Si cambia la selección leída, React puede actualizar el modal.

MEDIALOG · ELEGIR

La página comunica qué medio se eligió

src/pages/media-detail-page.tsx · acción disponible en éxito

```
const openCreate = useCreateMediaLogModalStore(
  (state) ⇒ state.openCreate
)

<Button
  onClick={() ⇒ openCreate(detailInput)}
>
  Agregar a mi biblioteca
</Button>
```

Clic → medio elegido

El clic ejecuta la función.

`detailInput` contiene el medio validado de la ruta.

No enviamos el evento del mouse al store.

MEDIALOG · ABRIR

El modal se monta una vez en PublicLayout

src/layouts/public-layout.tsx · posición dentro del layout

```
<>
  <AppHeader />

  <main>
    <Outlet />
  </main>

  <CreateMediaLogModal />
</>
```

Una estructura compartida

`Outlet` muestra búsqueda o detalle.

El modal queda fuera de la página hija.

Ambas páginas escriben en la misma selección.

MEDIALOG · COMPLETAR

El medio ya está elegido; completamos sus datos

Selección recibida

```
mediaType: movie
```

```
externalId: 123
```

El usuario no vuelve a escribir estos datos.

Ejemplo visual · formulario de creación

Agregar a mi biblioteca

Estado

Por ver

Prioridad

Media

Notas

Ver el fin de semana

Preparar body

MEDIALOG · VALIDAR

Un schema es una regla que podemos ejecutar

src/schemas/media-log.ts · reglas de los campos

```ts
import { z } from 'zod/v4'

export const mediaLogFormSchema = z.object({
  status: z.enum(MEDIA_STATUS_VALUES),

  priority: z.enum(MEDIA_PRIORITY_VALUES),

  notes: z.string().max(256),
})
```

Zod comprueba



`enum`: acepta un valor de una lista cerrada.

`max(256)`: limita la longitud de las notas.

Definir el schema no basta: alguna operación debe ejecutarlo.

MEDIALOG · VALIDAR

La misma regla, tres entradas distintas

Entrada a safeParse	Resultado	Qué comprobamos
pending · medium · «Ver después»	**success: true**	Datos disponibles en `result.data`
inventado · medium · «Ver después»	**success: false**	Estado fuera de la lista
pending · medium · 257 caracteres	**success: false**	Notas demasiado largas

Primero predecimos; después comprobamos.

MEDIALOG · COMPLETAR

useForm devuelve las herramientas del formulario

src/components/create-media-log-form.tsx · configuración inicial

```tsx
import { useForm } from '@mantine/form'

const form = useForm<MediaLogFormValues>({
  mode: 'uncontrolled',

  initialValues: {
    status: 'pending',
    priority: 'medium',
    notes: '',
  },
})
```

Entrada y salida

Mantine

`initialValues` configura el inicio.

`form` es el objeto devuelto.

Desde él usaremos `getInputProps` y `onSubmit`.

MEDIALOG · COMPLETAR

Escribir no obliga a renderizar todo el formulario

Controlado · lo que ya vimos

Un cambio actualiza estado React.

El render entrega un nuevo `value` al input.

Uncontrolled · Mantine Form

El input muestra lo que escribimos.

Form conserva los valores actuales en una referencia interna.

Uncontrolled no significa «sin validación» ni «React nunca vuelve a renderizar».

MEDIALOG · VALIDAR

El resolver conecta Zod con Mantine Form

src/components/create-media-log-form.tsx · opciones de useForm

```
import { schemaResolver } from '@mantine/form'

// Dentro de las opciones de useForm:
validate: schemaResolver(
  mediaLogFormSchema,
  { sync: true }
),

validateInputOnBlur: true,
```

Resolver = puente

Ejecuta las reglas y entrega errores a Form.

`sync: true`: las reglas terminan sin esperar una promesa.

`blur`: salir de un campo.

Se valida al salir del campo y al intentar preparar el body.

MEDIALOG · COMPLETAR

getInputProps conecta un campo con Form

src/components/create-media-log-form.tsx · JSX del campo

```
<Textarea
  label="Notas"
  key={form.key('notes')}
  {...form.getInputProps('notes')}
/>
```

Una conexión completa

Aporta valor inicial, eventos y error.

`...` distribuye esas propiedades en el componente.

No añadimos otro `value` u `onChange` que compita con Form.

MEDIALOG · COMPLETAR

La etiqueta se lee; el value viaja

src/components/create-media-log-form.tsx · opción de Select

```tsx
<Select
  label="Estado"
  data={[
    { value: 'pending', label: 'Por ver' },
    { value: 'watching', label: 'Viendo' },
  ]}
  allowDeselect={false}
  key={form.key('status')}
  {...form.getInputProps('status')}
/>
```

Dos lenguajes

`Persona: Por ver`

`Contrato: pending`

La etiqueta puede traducirse; el valor acordado se conserva.

Las listas de valores permitidos se reutilizan desde el contrato.

MEDIALOG · VALIDAR

onSubmit valida antes de ejecutar la preparación

Preparar body
La persona envía el formulario.

valida →

Reglas de Zod
Comprueba los tres campos.

si cumple →

Función de preparación
Recibe los valores válidos.

Si hay errores, se muestran en los campos y no se prepara un body nuevo.

MEDIALOG · PREPARAR

Form entrega los campos cuando son válidos

src/components/create-media-log-form.tsx · dentro del componente

```
const [preparedBody, setPreparedBody] =
  useState<CreateCatalogMediaLogBody | null>(null)

const handleSubmit = form.onSubmit((values) => {
  const body = toCreateMediaLogBody(
    identity,
    values
  )

  setPreparedBody(body)
})
```

Dos entradas conocidas

`identity`: tipo e ID del medio elegido.

`values`: campos entregados por Form.

`preparedBody`: estado local para mostrar el resultado.

El botón para confirmar dice «Preparar body». No guarda en el servidor.

MEDIALOG · PREPARAR

El adapter reúne los datos del medio y del formulario

Medio elegido

`movie · 123`

Viene de la selección.

se une a →

Formulario

pending · medium

« Ver después »

transforma →

Body

source: catalog

Datos del medio + campos

Notas limpias

Adapter = función que prepara datos para otro uso. Aquí construye el body, sin title, posterUrl ni userId.

MEDIALOG · PREPARAR

Vacío y espacios significan «sin notas»

src/adapters/media-log-form-to-create-body.ts · transformación

```
const notes = values.notes.trim()

// Propiedad del body:
notes: notes === '' ? null : notes
```

Campo	Body
`''`	`null`
`' '`	`null`
`' Ver después '`	`'Ver después'`

MEDIALOG · PREPARAR

La vista previa corresponde a lo último preparado

src/components/create-media-log-form.tsx · opción de useForm

```
onValuesChange: () ⇒ {
  setPreparedBody(null)
},
```

Al volver a editar

Ocultamos el body anterior.

La persona debe prepararlo otra vez.

Así no mostramos un resultado antiguo junto a campos nuevos.

El evento de cambio ya nos dice cuándo limpiar la vista previa.

MEDIALOG · ABRIR

Cerrar desmonta el formulario

src/components/create-media-log-modal.tsx · contenido del Modal

```
{selection !== null ? (
  <CreateMediaLogForm
    key={
      selection.mediaType + ':' +
      selection.externalId
    }
    identity={selection}
    onCancel={closeCreate}
  />
) : null}
```

Un nuevo comienzo

Cerrar asigna `selection: null`.

El formulario deja de renderizarse.

Reabrir crea una instancia con sus valores iniciales.

MEDIALOG · PREPARAR

Trabajo con agente

Objetivo

Redactar tests del adapter que ya escribimos.

Límites

Solo su archivo de test y fixtures dentro de `__tests__/`.

Lectura obligatoria

Revisar cada caso y el diff: entrada, resultado y razón.

Evidencia

Ejecutar los tests y explicar qué comprobaron.

La persona define el comportamiento antes de delegar.

FIN DE LA PRESENTACIÓN · SIGUE EL CODING

Ahora construimos el recorrido completo

01

Abrir

Dos páginas, una selección y un modal.

02

Validar

Estado, prioridad y notas con reglas visibles.

03

Preparar

Body local y reinicio comprobable.

Checkpoint esperado: datos válidos preparados; todavía sin creación remota.